

Agentic AI — Concepts Quick Review

A concise revision sheet based on the concepts covered so far.

1. Tokenization

Tokenization is the process of converting text into smaller units called **tokens**. Tokens can be whole words, sub-words, punctuation, or special symbols.

Why it matters: Token count affects context-window usage, API usage/cost, memory/history size, and processing.

Remember: Tokenization is broader than simply splitting words into sub-words.

Exam: Tokenization converts text into tokens that an LLM can process; token count influences context limits and commonly affects API cost.

2. Vectorization / Embeddings

Vectorization (more precisely, generating **embeddings**) converts text into numerical vectors in a high-dimensional vector space. Semantically related items tend to have similar/nearby representations.

Example: “cat” and “kitten” tend to be closer than “cat” and “car”. Similar does not mean identical.

RAG connection: Documents are chunked, embedded, stored in a vector database, and later retrieved using similarity search.

Exam: Embeddings represent semantic information numerically and enable similarity search, forming a key foundation of RAG.

3. Attention Mechanism

Attention allows a model to determine which tokens are relevant to a particular token and how strongly they are related. It helps capture contextual relationships and long-range dependencies.

Think: “Which other tokens should I pay attention to to understand this token?”

Agentic AI: Attention helps LLMs process instructions, conversation history, retrieved context, and tool outputs.

Exam: Attention captures dependencies between tokens by assigning different attention weights to relevant tokens.

4. Large Language Model (LLM)

An LLM is a large neural network, typically based on the Transformer architecture, pretrained on large-scale text data. A common pretraining objective is **next-token prediction**.

Example: “The cat is sitting on the ____” → the model predicts a likely next token such as “mat”.

Important: Pretraining teaches general language patterns; fine-tuning can later specialize the model.

Exam: An LLM is a neural network pretrained on large-scale data, commonly using next-token prediction to learn language patterns.

5. Autoregressive Generation

Autoregressive generation produces text **one token at a time**. Each generated token becomes part of the sequence used to predict the next token.

Flow: Prompt → predict token → append token → predict next token → repeat.

Important: Normal inference is **not training**. The model's weights are not updated during ordinary generation.

Exam: Autoregressive generation uses previously generated tokens as context for predicting subsequent tokens.

6. Fine-tuning

Fine-tuning is the process of further training a pretrained model on task-specific or domain-specific data so it performs better for that purpose.

Supervised Fine-Tuning (SFT): commonly uses input → desired-output examples.

Structured output: A model can be fine-tuned to follow a desired format, such as JSON, but structured output is not the definition of fine-tuning.

Exam: Fine-tuning adapts a pretrained model to a specific task/domain using additional training data.

7. RLHF — Reinforcement Learning from Human Feedback

RLHF aligns model behavior with human preferences. Humans compare/rank model responses; a reward model learns those preferences, and reinforcement learning optimizes the model toward higher-reward behavior.

Flow: Generate responses → human preference/ranking → reward model → reinforcement learning → improved behavior.

Important: RLHF is a specific approach involving human feedback; it is not synonymous with all reinforcement learning.

Exam: RLHF uses human preference data to train a reward model and optimize an LLM toward preferred outputs.

8. Chain of Thought (CoT)

Chain of Thought is a reasoning approach where a complex problem is handled through intermediate reasoning steps rather than jumping directly to the final answer.

Simple flow: Complex problem → intermediate steps → final answer.

Verification: Step-by-step reasoning can make errors easier to locate or verify, but it does not guarantee that a model will detect its own mistakes.

Do not confuse: CoT is not the same as agent planning. Planning is deciding which actions/tasks to perform to achieve a goal.

Exam: CoT uses intermediate reasoning steps to solve multi-step problems.

9. Reasoning Models

A reasoning model is designed to use additional inference-time computation to solve complex problems before producing its final response.

Examples: Reasoning-oriented models include DeepSeek-R1.

Distinction: CoT is a reasoning approach/technique; a reasoning model is a model optimized for stronger reasoning.

Exam: Reasoning models use additional inference-time computation for complex mathematical, logical, and coding tasks.

10. Quantization

Quantization reduces the numerical precision used to represent model values, especially weights. This can reduce memory consumption and inference cost and may improve deployment efficiency.

Example: FP32 = 32 bits, FP16 = 16 bits, INT8 = 8 bits, INT4 = 4 bits.

Important: Quantization can also be applied to activations and other values, not only weights. Lower precision can introduce some accuracy loss.

Easy memory: Same model, smaller numbers.

Exam: Quantization is a model-compression technique that reduces numerical precision to lower memory and computational requirements.

11. Knowledge Distillation

Knowledge distillation is a model-compression technique in which a smaller **student** model learns to mimic the outputs or behavior of a larger **teacher** model.

Flow: Large teacher → outputs/knowledge → smaller student → similar behavior with fewer parameters.

Easy memory: Big model teaches small model.

Difference from quantization: Quantization = same model, lower numerical precision. Distillation = train a smaller model to imitate a larger one.

Exam: Distillation transfers useful behavior from a teacher model to a smaller student model to reduce computational requirements.

■ Last-Minute Comparison

Concept	One-line memory hook
Tokenization	Text → tokens
Embeddings	Text → semantic vectors
Attention	Which tokens matter to each other?
LLM	Learns language patterns; commonly next-token prediction
Autoregressive generation	Predict token → append → predict next
Fine-tuning	Pretrained model → specialize
RLHF	Human preferences → reward → optimize
Chain of Thought	Complex problem → intermediate reasoning
Reasoning model	More inference-time computation for hard problems
Quantization	Same model, smaller numerical precision
Distillation	Big teacher teaches small student

Quick self-test before your meeting: Explain the difference between tokenization and embeddings; attention and CoT; fine-tuning and RAG; quantization and distillation; and autoregressive generation versus training.